Unió-Holvan

Szerző: Nikházy László
Előadó: Varga Péter

2026. március 4.

Partíció — definíció

Adott egy U alaphalmaz (pl. $\{1, 2, \ldots, n\}$).

Definíció: Egy **partíció** az U egy olyan felbontása

$$U = S_1 \cup S_2 \cup \cdots \cup S_k$$

ahol:

- $S_i \neq \emptyset$,
- $S_i \cap S_j = \emptyset$ ha $i \neq j$,
- minden elem pontosan egy S_i-ben szerepel.

Intuíció:

- az elemeket *diszjunkt halmazokba* csoportosítjuk,
- minden elemhez egy „címke” tartozik: melyik halmazban van.

Példa:

$$U = \{1, 2, 3, 4, 5, 6\}, \quad \{\{1, 4\}, \{2, 3, 6\}, \{5\}\}$$

Unió–Holvan — absztrakt adatszerkezet

Az **Unió–Holvan** (Disjoint Set Union, DSU) adatszerkezet egy **partíciót** tárol egy alaphalmazon.

Támogatott műveletek:

- **`holvan(x)`**: visszaadja, melyik halmazban van `x` (tipikusan egy halmazazonosítóval vagy gyökérelemmel).
- **`unio(x, y)`**: egyesíti azt a két halmazt, amelyben `x` és `y` vannak.

Formális megfogalmazás:

- A struktúra mindig egy partíciót reprezentál.
- `unio` két partícióosztályt összeolvaszt.
- `holvan` az ekvivalenciaosztály reprezentánsát adja vissza.

Unió–Holvan implementáció tömbbel (naiv)

Reprezentáció:

- Minden elemhez eltároljuk, melyik halmazban van.
- Egy tömb: `id[x]` = halmazazonosító.

Példa: Tegyük fel, hogy az elemek: `1, 2, 3, 4, 5, 6`

Unió előtt: Halmazok: `{1, 2}, {3, 4}, {5, 6}`

x	1	2	3	4	5	6
id[x]	1	1	3	3	5	5

unio(2,4): Minden `id[i]` = 3 elemet átírunk 1-re.

Unió után: Halmazok: `{1, 2, 3, 4}, {5, 6}`

x	1	2	3	4	5	6
id[x]	1	1	1	1	5	5

Naiv tömbös implementáció: műveletek és komplexitás

Műveletek:

`holvan(x)`:

- egyszerűen visszaadjuk `id[x]`-et,

`unio(x, y)`:

- `a` = `id[x]`, `b` = `id[y]`,
- ha `a` = `b`, nincs teendő,
- különben minden `i`-re, ahol `id[i] = b`, átírjuk `id[i] = a`.

Komplexitás:

- `holvan(x)`: `O(1)`
- `unio(x,y)`: `O(n)` (teljes tömb átfutása)

Hátrány: `O(n)` unió esetén $O(n^2)$ összidő.

Unió–Holvan implementáció tömbbel és listákkal

Reprezentáció:

- Minden elemhez: `id[x]` = halmazazonosító.
- Minden halmazhoz: lista az elemeiről.

$$\texttt{lista[h]} = \{\texttt{x} \mid \texttt{id[x]} = \texttt{h}\}$$

Példa: elemek: `1, 2, 3, 4, 5, 6`

Halmazok:

$$\texttt{lista[1]} = \{1, 2\}, \quad \texttt{lista[3]} = \{3, 4\}, \quad \texttt{lista[5]} = \{5, 6\}$$

id tömb:

x	1	2	3	4	5	6
id[x]	1	1	3	3	5	5

Tömb + listák: műveletek és komplexitás

Műveletek:

`holvan(x)`:

- visszaadjuk `id[x]`-et.

`unio(x, y)`:

- $a = $ `id[x]`, $b = $ `id[y]`,
- ha $a = b$, nincs teendő,
- különben minden $v \in$ `lista[b]`-re:
  - `id[v] = a`
  - hozzáadjuk v-t `lista[a]`-hoz,
- `lista[b]` törölhető.

Komplexitás:

- `holvan(x)`: $O(1)$
- `unio(x,y)`: $O(|\text{halmaz}_b|)$

Small-to-large merging és amortizált futásidő

Small-to-large merging (kisebb a nagyobba):

- unió során mindig a **kisebb halmaz elemeit** másoljuk át a nagyobba,
- a nagyobb halmaz marad a reprezentáns.

Miért gyors?

- Ha egy elem átkerül egy új halmazba, annak mérete **legalább kétszereződik**.
- Egy halmaz mérete legfeljebb n.
- Egy elem legfeljebb $\log_2 n$-szer tud halmazt váltani.

Összesített futásidő:

$$O(n \log n)$$

Amortizált költség fogalma:

- Nem egyetlen műveletet, hanem **sok művelet átlagos költségét** vizsgáljuk.
- Bár egy unió drága lehet, egy elem összesen keveset mozog.
- Így egy unió művelet átlagosan: $O(\log n)$.

Unió–Holvan implementáció fákkal

Reprezentáció:

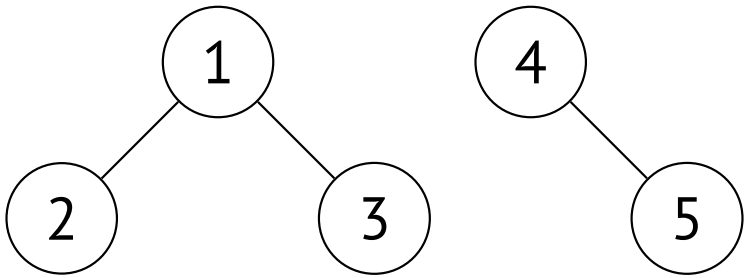
- minden elemhez eltároljuk a szülőjét: `szulo[x]`,
- ha `szulo[x] = x`, akkor `x` egy **gyökér**,
- egy halmaz azonosítója a fa gyökere.

holvan(x) = a fa gyökerének megkeresése.

unio(x,y):

- megkeressük a gyökereket,
- az egyik gyökeret a másik alá kötjük.

Fa-reprezentáció szemléltetése



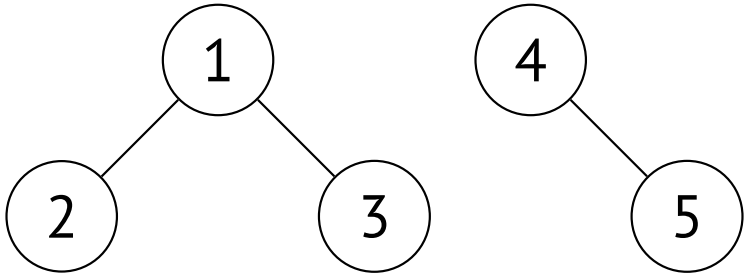
Halmazok:

- Gyökér 1: {1,2,3}
- Gyökér 4: {4,5}

szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	4	4

unio(3,5) – unió előtt



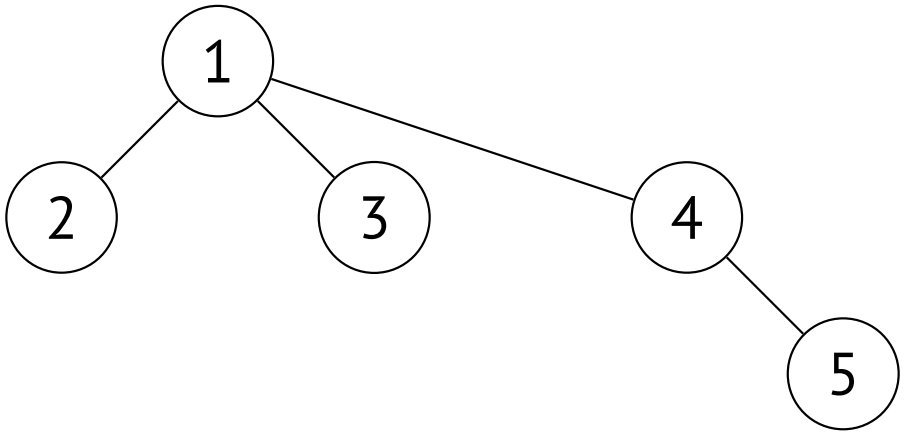
Gyökerek:

$$\texttt{holvan(3)} = 1, \quad \texttt{holvan(5)} = 4$$

szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	4	4

unio(3,5) – unió után



szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	1	4

holvan(5): $5 \rightarrow 4 \rightarrow 1$

Fa-alapú Unió–Holvan: műveletek és komplexitás

holvan(x):

- felfelé megyünk a szülőkön, amíg gyökeret nem találunk,
- költség: O(fa mélysége).

unio(x,y):

- két holvan hívás + egy él hozzáadása,
- költség: O(fa mélysége).

Probléma:

- rossz unió sorrendben a fa lehet lánc,
- mélység = $O(n)$.

Rang szerinti unió

Ötlet:

- minden gyökérhez tároljuk a **rangot** (fa becsült mélységét),
- kisebb rangú fát kötjük a nagyobb rangú alá.

Rang frissítése:

- kezdetben minden fa rangja 1.
- ha két azonos rangú fát uniózunk,
- az új gyökér rangja nő 1-gyel.

Eredmény:

fa mélysége $O(\log n)$

Méret szerinti unió

Ötlet:

- minden gyökérhez tároljuk a halmaz méretét,
- kisebb elemszámú fát kötjük a nagyobba.

Mélység elemzés:

- ha egy elem mélysége nő, a halmazméret legalább kétszereződik,
- ezért egy elem legfeljebb $\log n$-szer kerül mélyebbre.

Komplexitás:

$O(\log n)$ műveletenként

Kód (méret szerinti unió)

```
1  vector<int> szulo, meret;
2
3  void init(int n) {
4    meret.assign(n + 1, 1);
5    szulo.resize(n + 1);
6    iota(szulo.begin(), szulo.end(), 0);
7  }
8
9  int holvan(int x) {
10   while (szulo[x] != x)
11     x = szulo[x];
12   return x;
13 }
14
15 void unio(int a, int b) {
16   a = holvan(a);
17   b = holvan(b);
18   if (a == b) return;
19   if (meret[a] < meret[b]) swap(a, b);
20   szulo[b] = a;
21   meret[a] += meret[b];
22 }
```

Útösszenyomás (path compression)

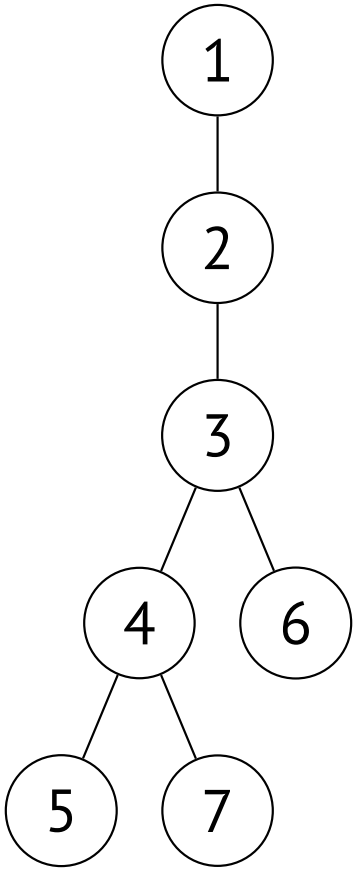
Ötlet:

- `holvan(x)` közben minden csúcsot közvetlenül a gyökérhez kötünk.
- A fa fokozatosan **ellaposodik**.

Hatás:

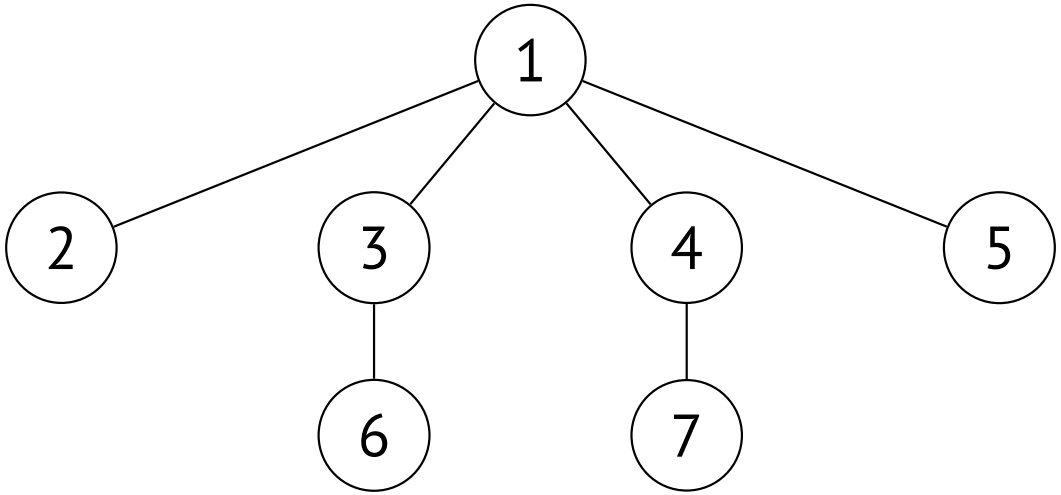
- későbbi `holvan` hívások nagyon gyorsak,
- a fa gyakorlatilag csillag alakúvá válik.

Útösszenyomás előtt



holvan(5): $5 \to 4 \to 3 \to 2 \to 1$
holvan(6): $6 \to 3 \to 2 \to 1$
holvan(7): $7 \to 4 \to 3 \to 2 \to 1$

Útösszenyomás után



holvan(5): $5 \to 1$
holvan(6): $6 \to 3 \to 1$
holvan(7): $7 \to 4 \to 1$

Sok további csúcs útja is rövidült, egyedül a 6-os és 7-es nem közvetlenül a gyökérhez kötődnek.

Komplexitás útösszenyomással (bizonyítás nélkül)

Csak útösszenyomással:

amortizált költség $O(\log n)$

Méret/rang szerinti unió + útösszenyomás:

$$O(\alpha(n))$$

Inverz Ackermann függvény ($\alpha(n)$):

- rendkívül lassan nő,
- ha $n < 10^{600}$, akkor $\alpha(n) \leq 4$,
- gyakorlatilag **konstans**.

Unió–Holvan kód: méret + útösszenyomás

```
1   vector<int> szulo, meret;
2
3   void init(int n) {
4       meret.assign(n + 1, 1);
5       szulo.resize(n + 1);
6       iota(szulo.begin(), szulo.end(), 0);
7   }
8
9   int holvan(int x) {
10      if (szulo[x] == x) return x;
11      return szulo[x] = holvan(szulo[x]);  // útösszenyomás
12  }
13
14  void unio(int a, int b) {
15      a = holvan(a);
16      b = holvan(b);
17      if (a == b) return;
18      if (meret[a] < meret[b]) swap(a, b);
19      szulo[b] = a;
20      meret[a] += meret[b];
21  }
```

Alkalmazások

Road Construction - Feladat

`https://cses.fi/problemset/task/1676/`

Adott n város $(1 \leq n \leq 10^5)$, kezdetben nincs köztük út. m napon keresztül minden nap egy új utat építenek $(1 \leq m \leq 2 \cdot 10^5)$.

Feladat: minden napra add meg, hogy

- hány összefüggő komponens van (városcsoport, ahol bármelyik két város között van út),
- hány városból áll a legnagyobb komponens.

Bemenet: Első sor: n m. Következő m sor: a b - új út épül a a és b városok között.

Kimenet: Minden nap egy sor: komponensek száma, legnagyobb komponens mérete.

Példa bemenet:

```
5 3
1 2
1 3
4 5
```

Kimenet:

```
4 2
3 3
2 3
```

Road Construction - Elvi megoldás

Az **Unió-Holvan** ideális adatszerkezet **gráf komponenseinek nyilvántartására**.

Minden egyes útnál tudjuk frissíteni a komponensek számát és a legnagyobb komponens méretét.

Az unio(a,b) műveletben:

- ha két különböző komponenst egyesítettünk, akkor csökkentjük a komponensek számát
- frissítjük a legnagyobb komponens méretét, ha a létrejövő komponens nagyobb nála
- ez konstans extra művelet az unio(a,b)-n belül

Komplexitás: $O(n + m \cdot \alpha(n))$.

Road Construction - Kód

```
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int n, m, kdb, mx;
5   vector<int> szulo, meret;
6
7   int holvan(int x) {
8       if (szulo[x] == x) return x;
9       return szulo[x] = holvan(szulo[x]);
10  }
11  void unio(int a, int b) {
12      a = holvan(a);
13      b = holvan(b);
14      if (a == b) return;
15      if (meret[a] < meret[b]) swap(a, b);
16      szulo[b] = a;
17      meret[a] += meret[b];
18      kdb--;
19      mx = max(mx, meret[a]);
20  }
21
22  int main() {
23      ios::sync_with_stdio(false); cin.tie(nullptr);
24      cin >> n >> m;
25      szulo.resize(n+1); meret.assign(n+1, 1);
26      iota(szulo.begin(), szulo.end(), 0);
27      kdb = n; mx = 1;
28      for (int i = 0; i < m; i++) {
29          int a, b; cin >> a >> b;
30          unio(a, b);
31          cout << kdb << "␣" << mx << "\n";
32      }
33  }
```

Consecutive Letters - Feladat

`https://www.spoj.com/problems/CONSEC/`

Adott egy `S` hosszú string $(1 \leq |S| \leq 200\,000)$, amely csak nagybetűket tartalmaz. Két típusú lekérdezést kapunk:

- `1 i`: Határozd meg a maximális hosszát annak a `[b, e]` intervallumnak, melyre $b \leq i \leq e$ és a `S[b...e]` substring csak `S[i]` betűt tartalmaz.
- `2 i`: Cseréld a `i`-edik karaktert #-re.

Mindkét típusú lekérdezésnél garantált, hogy $S[i] \neq \#$.

Bemenet:
Első sor: maga a string `S`. Következő sor: `Q`, a kérdések száma. Következő `Q` sor: minden sor egy query, formátum: `1 i` vagy `2 i`

Példa bemenet:

```
AABBBCCCC
5
1 0
2 1
1 0
2 2
1 3
```

Kimenet:

```
2
1
2
```

Consecutive Letters - Elvi megoldás

Ötlet: a karakterekből álló, összefüggő szegmenseket halmazokként kezeljük:

- Minden halmazhoz tároljuk a hosszát.
- `holvan(i)` segítségével megtaláljuk, melyik halmazhoz tartozik az `i`-edik karakter.
- `1 i` lekérdezés: visszaadjuk a halmaz hosszát.
- `2 i` lekérdezés (karakter törlése #-re):
  - A törölt karakter miatt a halmaz felhasadhat két részre.
  - A felosztást nem tudjuk hatékonyan csinálni.
  - **Ötlet:** A műveleteket **hátulról visszafelé is elvégezhetjük**, ekkor a szegmensek egyesítése történik $\rightarrow$ Unió–Holvannal tökéletesen kezelhető a gyors összefűzés és lekérdezés.

Consecutive Letters - kód

```
1  string s; cin >> s;
2  int n = s.size(), q; cin >> q;
3  vector<pair<int,int>> kerdesek(q);
4  string akt = s; // másolat a törlésekhez
5  for (auto& [t, idx] : kerdesek) {
6      cin >> t >> idx;
7      if (t == 2) akt[idx] = '#';
8  }
9  init(n);
10 for (int i = 1; i < n; i++) {
11     if (akt[i] != '#' && akt[i] == akt[i-1]) unio(i-1, i);
12 }
13 vector<int> megoldas;
14 reverse(kerdesek.begin(), kerdesek.end());
15 for (auto [t, idx] : kerdesek) {
16     if (t == 1) {
17         megoldas.push_back(meret[holvan(idx)]);
18     } else {
19         akt[idx] = s[idx];
20         if (idx > 0 && akt[idx-1] == akt[idx]) unio(idx-1, idx);
21         if (idx + 1 < n && akt[idx+1] == akt[idx]) unio(idx, idx+1);
22     }
23 }
24 reverse(megoldas.begin(), megoldas.end());
```

Legkisebb kizárt (MEX) - Feladat

A feladat leírása: Tartsunk nyilván természetes számok egy halmazát, amelybe folyamatosan érkeznek elemek. Két műveletet kell hatékonyan megvalósítanunk:

- **`Adat(x)`**: Az `x` természetes szám bekerül a halmazba (ha már benne volt, nem történik semmi).
- **`Mex(a)`**: Mi az a legkisebb `b` szám, amelyre $a \leq b$, és `b` **nincs** benne az eddigi halmazban?

Példa: Ha a halmaz: `{0, 1, 2, 4, 5}`

- `Mex(0)` $\rightarrow$ 3
- `Mex(4)` $\rightarrow$ 3 helyett 6 (mivel $b \geq a$)
- `Mex(7)` $\rightarrow$ 7

Korlátok:

- Értékek: $0 \leq x \leq 10^7$
- Műveletek száma: 10^5

Megoldás: Intervallum-ugrás Unió-Holvannal

Az alapötlet: Az egymást követő, már beérkezett számokat fogjuk össze egy-egy halmazba. Minden halmaz reprezentánsa a **szakasz utáni első szabad szám** legyen.

Algoritmus:

- **Inicializálás:** Minden `i` számra `szulo[i] = i`.
- **`Adat(x)`**: Ha `x` bekerül, vonjuk unióba `x`-et a szomszédos $x + 1$ elemmel: `unio(x, x+1)`.
- **Irányított unió:** A megvalósításnál figyelni kell, hogy mindig a **nagyobb** indexű elem legyen az új szülő:
  - `szulo[holvan(x)] = holvan(x+1)`
- **`Mex(a)`**: A válasz egyszerűen a halmaz reprezentánsa: `holvan(a)`.
- **Fontos:** használjunk **útösszenyomást** a holvan függvényben, hogy gyors legyen a megoldás (az uniónál nincs választásunk).

Legkisebb kizárt - kód

```
1  const int M = 10000005;
2  int szulo[M];
3  bool inicializalva = false;
4
5  void init() {
6      if (inicializalva) return;
7      inicializalva = true;
8      iota(szulo, szulo +  M, 0);
9  }
10
11 int holvan(int x) {
12     if (szulo[x] == x) return x;
13     return szulo[x] = holvan(szulo[x]);
14 }
15
16 void Adat(int x) {
17     init();
18     szulo[holvan(x)] = holvan(x + 1);
19 }
20
21 int Mex(int a) {
22     init();
23     return holvan(a);
24 }
```

Maximum-keresés dinamikus intervallumon

Matematikai megfogalmazás: Adott $c_1, \ldots, c_n$ és $l_1, \ldots, l_n$ sorozatok (ahol $0 \leq l_i < i$). Számítsuk ki minden $i = 1 \ldots n$ esetén az alábbi a_i értéket:

$$a_i = c_i + \max_{l_i \leq j < i} a_j$$

ahol $a_0 = 0$.

Kihívás:

- A naiv keresés $O(n^2)$ lenne, ami nagy n esetén túl lassú.
- Az intervallum kezdőpontja (l_i) nem monoton, így a klasszikus csúszóablakos maximum (monoton sor) nem alkalmazható közvetlenül.

Megoldási irány: Olyan adatszerkezetre van szükségünk, amely támogatja az elemek hatékony törlését és a legközelebbi "élő" elem megtalálását.

Megoldás: Unió-Holvan és Monoton sor

Az algoritmus elve: Tartsunk fenn egy indexekből álló listát, ahol az a_j értékek szigorúan monoton csökkennek.

1. **Törlés (Unió):** Ha egy új a_i kiszámítása után kiderül, hogy $a_i \geq a_j$, akkor a korábbi j index "feleslegessé" válik. Ahelyett, hogy ténylegesen törölnénk a listából, az Unió-Holvannal minden felesleges indexet a tőle jobbra lévő érvényes indexhez kötünk.
2. **Lekérdezés (Holvan):** Az l_i indexről indulva a $\texttt{holvan}(l_i)$ megadja az első olyan indexet jobbra, amely a monoton sor része.

Strange Food Chain (SPOJ) - Feladat

Alaphelyzet: Három állatfaj van (A, B, C), amelyek körbeeszik egymást: $\mathtt{A} \to \mathtt{B} \to \mathtt{C} \to \mathtt{A}$. Adott $\mathtt{n}$ állat és $\mathtt{k}$ információ róluk.

Információk típusai:

- **1 x y:** $\mathtt{x}$ és $\mathtt{y}$ azonos fajba tartozik.
- **2 x y:** $\mathtt{x}$ megeszi $\mathtt{y}$-t.

Egy információ hamis, ha:

1. $\mathtt{x}$ vagy $\mathtt{y} > \mathtt{n}$.
2. Azt állítja, hogy $\mathtt{x}$ megeszi saját magát.
3. **Ellentmond** egy korábbi, már igaznak elfogadott állításnak.

Feladat: Számoljuk meg, hány állítás hamis a $\mathtt{k}$ darabból!

Korlátok: $\mathtt{n} \leq 50\,000$, $\mathtt{k} \leq 100\,000$.

Megoldás: Relációs Unió-Holvan

A trükk: Minden elemhez tároljuk a kapcsolatát a szülőjével a táplálékláncban! Legyen `tav[x]` az `x` és a `szulo[x]` kapcsolata:

- **0:** azonos fajba tartoznak.
- **1:** `x` megeszi a szülőt (`A` $\rightarrow$ `B`).
- **2:** a szülő eszi `x`-et (`B` $\rightarrow$ `A`).

Műveletek:

- **Holvan(x):** Útösszenyomáskor a távolságokat összeadjuk mod 3. Így megkapjuk `x` relációját az aktuális gyökérhez képest.
- **Unió(x, y):** Ha két állat már egy halmazban van, ellenőrizzük a távolságaik különbségét (mod 3). Ha nem egyezik az új információval $\rightarrow$ hamis.
- Ha nincsenek egy halmazban, egyesítjük a két fát, és kiszámoljuk az új gyökerek közötti távolságot.

Strange Food Chain — Kód részlet

```
1   int szulo[50005], tav[50005];
2
3   int holvan(int x) {
4       if (szulo[x] == x) return x;
5       int regi_szulo = szulo[x];
6       szulo[x] = holvan(szulo[x]);
7       // Utosszenyomas: a tavolsagok mod 3 osszeadodnak
8       tav[x] = (tav[x] + tav[regi_szulo]) % 3;
9       return szulo[x];
10  }
11
12  bool unio(int d, int x, int y) {
13      int gy_x = holvan(x), gy_y = holvan(y);
14      int elvart_rel = d - 1; // 0: azonos, 1: x eszi y-t
15      if (gy_x == gy_y) {
16          // Ellenorzes: (x -> gy) - (y -> gy) mod 3 meg kell egyezzen d-1-gyel
17          return (tav[x] - tav[y] + 3) % 3 == elvart_rel;
18      }
19      szulo[gy_y] = gy_x;
20      tav[gy_y] = (tav[x] - tav[y] - elvart_rel + 6) % 3;
21      return true;
22  }
```

Unió–Holvan visszagörgetéssel (DSU with rollback)

Motiváció: Bizonyos feladatokban (pl. dinamikus gráfok, offline lekérdezések) nemcsak egyesíteni, hanem **vissza is kell vonni** unió műveleteket.

Ötlet:

- Minden `unio` művelet előtt eltároljuk a módosított változókat.
- Egy **történet verem (stack)** struktúrába mentjük a régi állapotokat.
- A `visszagörget` művelet a mentett értékek alapján visszaállítja az adatokat.

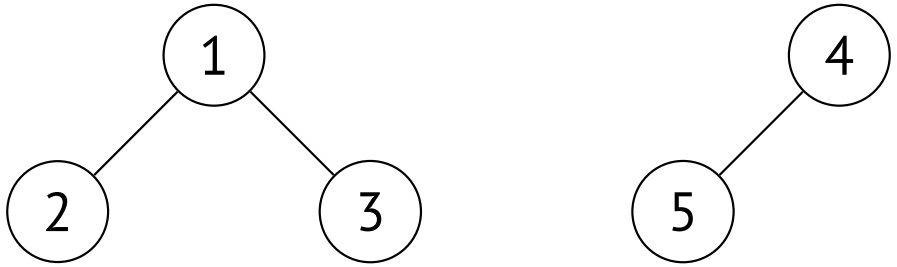
Pillanatkép (snapshot):

- Egy egész szám, amely megadja a történelem verem aktuális méretét.
- Később ide visszagörgetve pontosan ezt az állapotot kapjuk vissza.

Fontos: Visszagörgetéses Unió-Holvan esetén **NEM használunk útösszenyomást**, mert az túl sok állapotot változtatna.

Szemléltetés: unió művelet előtt

Kiinduló állapot:



szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	4	4

meret tömb:

x	1	2	3	4	5
meret[x]	3	1	1	2	1

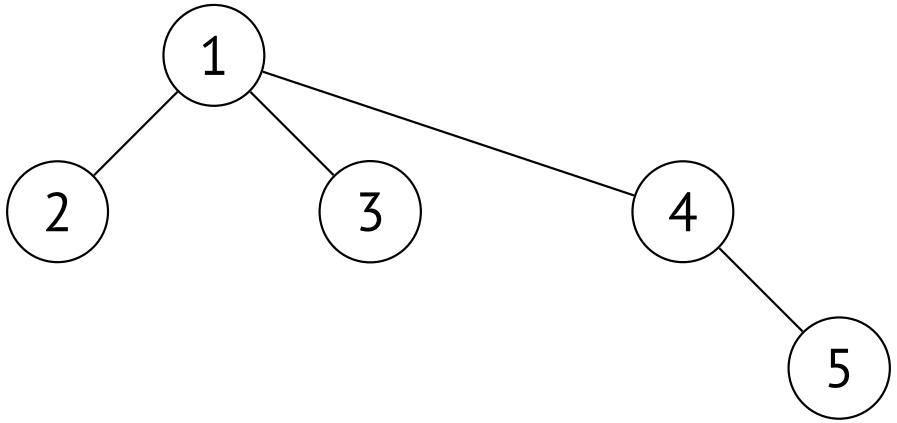
unio(3,5): a 3-as gyökere 1, az 5-ös gyökere 4, tehát 4-et kötjük 1 alá.

A verembe mentjük:

$$(\texttt{szulo}[4] = 4),\ (\texttt{meret}[1] = 3)$$

Szemléltetés: unió művelet után

Unió után:



szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	1	4

meret tömb:

x	1	2	3	4	5
meret[x]	5	1	1	2	1

Történet verem:

szulo[4] = 4
meret[1] = 3

Visszagörgetés: a veremből visszaállítjuk ezeket az értékeket.

Unió–Holvan visszagörgetéssel - kód

```
1   struct UnioHolvan {
2       vector<int> szulo, meret;
3       vector<pair<int*, int>> tortenet; // pointer + régi érték
4
5       UnioHolvan(int n) {
6           szulo.resize(n+1);
7           meret.assign(n+1, 1);
8           iota(szulo.begin(), szulo.end(), 0);
9       }
10      int holvan(int x) {
11          while (szulo[x] != x) x = szulo[x];
12          return x;
13      }
14      void unio(int a, int b) {
15          a = holvan(a); b = holvan(b);
16          if (a == b) return;
17          if (meret[a] < meret[b]) swap(a, b);
18          // mentjük a változtatásokat
19          tortenet.push_back({&szulo[b], szulo[b]});
20          tortenet.push_back({&meret[a], meret[a]});
21          szulo[b] = a;
22          meret[a] += meret[b];
23      }
24      int pillanatkep() {
25          return tortenet.size();
26      }
27      void visszagorget(int id) {
28          while (tortenet.size() > id) {
29              *tortenet.back().first = tortenet.back().second;
30              tortenet.pop_back();
31          }
32      }
33  };
```

Dynamic Connectivity (Dinamikus összefüggőség)

`https://cses.fi/problemset/task/2133/`

Adott egy irányítatlan gráf `n` csúccsal és `m` éllel. `k` esemény történik:

- **1** `a b`: Új él jön létre `a` és `b` között.
- **2** `a b`: Egy létező él `a` és `b` között megszűnik.

Feladat: Írd ki az összefüggő komponensek számát a legelső esemény előtt, majd minden egyes esemény után.

Korlátok:

- $n, m, k \leq 10^5$

Példa bemenet:

```
5 3 3
1 4
2 3
3 5
1 2 5
2 3 5
1 1 2
```

Kimenet:

```
2 2 2 1
```

Dynamic Connectivity - Megoldási elv

Offline feldolgozás (Divide and Conquer on Queries): Mivel az éltörlést az Unió-Holvan nem támogatja hatékonyan, az időt használjuk extra dimenzióként.

1. **Élettartam meghatározása:** Minden élhez rendeljünk intervallumokat $[t_{start}, t_{end}]$, amíg a gráfban van.
2. **Szegmensfa az idő felett:** Építsünk egy fát a $[0, k]$ időintervallumra. Minden élt szúrjunk be a fának azon csúcsaiba, amelyek lefedik az élettartamát ($O(\log k)$ csúcs).
3. **Bejárás (DFS):**
   - Lefelé haladva: adjuk hozzá az aktuális csomópontban tárolt éleket a **visszagörgethető Unió-Holvanhoz**.
   - Levélhez érve: az Unió-Holvanban lévő komponensek száma a válasz az adott időpillanatra.
   - Felfelé haladva: **visszagörgetjük** (rollback) az Unió-Holvant a csomópontba lépés előtti állapotra.

Komplexitás: $O((m + k) \cdot \log k \cdot \log n)$.

Feladatok

Önálló munkára

New Roads Queries (CSES)

`https://cses.fi/problemset/task/2101/`

Adott `n` város és `m` út, amelyek a megadott sorrendben épülnek meg. Válaszolj meg `q` lekérdezést!

Lekérdezés:

- a b: Mi az a legkorábbi időpillanat (nap), amikor az `a` és `b` városok között létrejön az összeköttetés?

Korlátok:

- `n, m, q` $\leq 2 \cdot 10^5$

Bemenet:

- `n m q`, majd `m` sorban az utak `(u, v)`, végül `q` sorban a kérdések.

Kimenet: Minden lekérdezésre egy szám: a nap, amikor a két város először ugyanabban a komponensben lesz, vagy -1, ha ez sosem történik meg.

Two Sets (Codeforces)

`https://codeforces.com/contest/468/problem/B`

Adott n különböző pozitív egész szám, valamint két szám: a és b. A feladat a számokat két halmazba (A és B) szétosztani úgy, hogy:

- Ha $x \in A$, akkor $(a - x)$ is legyen A eleme.
- Ha $x \in B$, akkor $(b - x)$ is legyen B eleme.

Kimenet:

- Ha nincs ilyen szétosztás: NO
- Ha van: YES és egy n hosszú 0/1 sorozat (melyik elem melyik halmazba került).

Korlátok:

- $n \leq 10^5$
- $a, b, x_i \leq 10^9$

BOI 2016 Park

`https://oj.uz/problem/view/BOI16_park`

Egy téglalap alakú parkban ($w \times h$) n darab kör alakú oszlop található. Egy látogató adott r sugarú kör alakú tárggyal szeretne áthaladni a parkon a négy sarokbejárat valamelyike között.

Lekérdezés:

- r sugár és e kezdő sarok esetén melyik sarkokba juthat el a látogató?

Akadályok:

- Két oszlop között akkor nem fér el, ha a távolságuk $< 2r$.
- Egy oszlop és a fal között akkor nem fér el, ha a távolságuk $< 2r$.

Korlátok:

- $n \leq 2000$, $m \leq 10^5$ lekérdezés.
- Az oszlopok nem érnek egymáshoz és nem lógnak ki a parkból.

Lekötő feladatok

BOI 2020 Joker
https://codeforces.com/contest/1386/problem/C